Programação de Computadores

Aula #07

Testando seus programas

Como testar seus programas de forma automática

Ciência da Computação - BCC e IBM
Prof. Andrey Ricardo Pimentel

Biblioteca assert.h

void assert(int expression)

A função assert() testa se uma expressão é verdadeira. Ao executar, se a expressão for verdadeira, o programa continua executando normalmente. Entretanto, se a expressão for falsa, o programa será interrompido e uma mensagem de erro será exibida. O uso da função acrescenta uma sobrecarga à execução do programa e ao tamanho do código gerado.

Recebe: uma expressão para ser avaliada.

Retorna: não há valor de retorno.

Biblioteca assert.h

```
//EXEMPLO ASSERT()
#include <stdio.h>
#include <assert.h>
int main() {
   int i;
   for (i=3; i<=10; i++) {
      assert(i <= 7);
      printf("i = %d\n", i);
   }
   return 0;
}
```

Biblioteca Assert.h

```
>clang-7 -pthread -lm -o main main.c
>./main
i = 3
i = 4
i = 5
i = 6
i = 7
main: main.c:9: int main(): Assertion 'i <= 7' failed.
exited, aborted
```

TDD em C

Entendo a importância de não reinventar a roda, mas dependemos muito rapidamente de frameworks de teste como JUnit e Jest sem questionar se eles são realmente essenciais.
Existem outros frameworks como Unity para C, mas vamos voltar ao básico e tentar uma abordagem sem frameworks.
No fim das contas, para escrever e executar testes automatizados, precisamos apenas de alguns conceitos:

TDD em C

Isolamento de testes: cada teste é uma função que se limpa após a execução;

Asserções: podemos definir expectativas que interrompem a execução do teste em caso de falha; além disso, podemos criar asserções personalizadas — matchers;

Conjunto de testes: crie grupos lógicos de testes;

Executor de testes: podemos executar (localmente e em CI/CD) os conjuntos de testes e ser notificados em caso de falhas.

TDD em C

```c
// includes ommited for brevity

// dummy test to prove everything is adequately setup
void test_addTodo() {
  assert(true);   // we need assert.h
}

// TODO: more tests will be added here

int main() {
 test_addTodo();
 // TODO: more tests will be invoked here

 return 0; // exit code 0 - means test suite success
}
```

TDD em C

```c
#include "todo_storage.h"

void test_addTodo() {
  ToDos todos = {0};

  int result = addToDoItem(&todos, "buy bigger shoes");
  // expected value is followed by actual (like in JUnit or similar)
  // on assert fail, program will exit immediately
  assert(0 == result);        // 0 represents no errors
  assert(1 == todos.size);
  assert(0 == strcmp("buy bigger shoes", todos.list[0].text)); // equal strings
}

int main() {
  test_addTodo();
  return 0;
}
```

TDD em C

Executando testes

Para executar nosso novo conjunto de testes, tudo o que precisamos fazer é:

```
# we need the module under test code and the test code

gcc todo_storage.c test_todo_storage.c -o test_todo_storage.exe

./test_todo_storage.exe
```

TDD em C

Se o conjunto de testes for aprovado, o código de saída será zero. Se o teste falhar, a execução é interrompida imediatamente e a saída indica onde (arquivo, função e linha de código) o teste falhou. Além disso, o código de saída diferente de zero permite a detecção antecipada de falhas, tanto localmente quanto no pipeline de CI/CD

```
Assertion failed: (1 == todos.size), function test_addTodo, file
test_todo_storage.c, line 6.
```

TDD em C

```
bool equalInts(const int expected, const int actual) {
    if (expected != actual) {
        fprintf(stderr, "❌ Ints are not equal\nExpected: %d\nActual:   %d\n",
expected, actual);
        return false;
    }
    return true;
}


bool equalStrings(const char* expected, const char* actual) {
    if (strcmp(expected, actual) != 0) {
        fprintf(stderr, "❌ Strings are not equal.\nExpected: \"%s\"\nActual:
\"%s\"\n", expected, actual);
        return false;
    }
    return true;
}
```

TDD em C

```c
void test_addTodo() {
  ToDos todos = {0};

  int result = addToDoItem(&todos, "buy bigger shoes");

  // compare this with the previous snippet:
  assert(equalInts(0, result));
  assert(equalInts(1, todos.size));
  assert(equalStrings("buy bigger shoes", todos.list[0].text));
}
```

TDD em C

Lidando com dependências

Imagine que você queira testar o módulo A, que depende do módulo B. Eu prefiro uma abordagem vertical para testes, onde você isola comportamentos e considera o uso de versões internas e simulações. No entanto, às vezes, você precisa isolar o módulo A por algum motivo.

Para fazer isso, você pode:

Criar um arquivo de teste para o módulo A (test_module_a.c).

Nele, inclua o arquivo de cabeçalho do módulo B (#include "module_b.h").

Criar um arquivo de teste para o módulo B (module_b_test_double.c), onde você implementa uma substituição da dependência de produção.

Aqui, você pode recorrer a versões fictícias (dummies), stubs, spies, mocks e fakes. Você não precisa de bibliotecas para isso.

TDD em C

Testes parametrizados

Os testes parametrizados (testes orientados a dados) são padrão em qualquer framework de testes.

Testar variações de dados (entrada e saída esperada) usando o mesmo caso de teste é prático. No entanto, devem ser usados com moderação para manter a legibilidade do teste.

Como fazer isso em C? Podemos injetar diferentes entradas no mesmo teste, uma de cada vez:

TDD em C

```
// you could inject more things like the expected size
void test_addTodo(const char* input) {
    ToDos todos = {0};
    addToDoItem(&todos, input);
    assert(1 == todos.size);
    assert(0 == strcmp(input, todos.list[0].text));
}
int main() {
    test_addTodo("buy bigger shoes"); // regular case
    test_addTodo("");                 // empty string
    test_addTodo("😀 ✅",);           // emojis
    test_addTodo("buy\t\nshoes"),     // mixed with escape characters
    // ...
    // we could have done with a for, but what's the benefit?
return 0;
}
```

Exercício #07B

Construa uma lista com, no máximo 100 elementos inteiros, baseada em array, e as funções para inserir no início e remover do início, retornar o tamanho da lista entre outras. Elabore e execute um conjunto de testes para as funções da lista.

Obrigado!

Andrey Ricardo Pimentel
inf.ufpr.br/andrey
andrey@inf.ufpr.br